



NODE.JS CHEAT SHEET

Core setup, initialization, operations & integration quick reference

PART 1: SETUP & CORE

01. SETUP & MODULES

Installation

```
# CommonJS (CJS):
const fs = require('fs')
module.exports = { fn }

# ESM (package.json: "type":"module"):
import fs from 'fs'
export const fn = () => {}

node --version / nvm use 20
```

04. FILE SYSTEM

Workflow

```
import { promises as fs } from 'fs'
const data = await fs.readFile('./file.txt', ...
await fs.writeFile('./out.txt', data)
await fs.mkdir('./dir', { recursive: true })
const files = await fs.readdir('./dir')
await fs.rename(old, new) / fs.unlink(path)
```

Always prefer async (promises) over sync methods

02. NPM BASICS

Architecture

```
npm init -y
npm install express (saves to dependencies)
npm install -D jest (saves to devDependencies)
npm run start / npm test
npm ci (clean install from package-lock.json)
npx create-next-app (run without install)
```

05. EVENTS (EVENTEMITTER)

CRUD API

```
import { EventEmitter } from 'events'
const emitter = new EventEmitter()
emitter.on('data', (payload) => { ... })
emitter.once('end', () => cleanup())
emitter.emit('data', { id: 1 })
emitter.removeListener('data', handler)
emitter.setMaxListeners(20) // avoid memory l...
```

03. HTTP SERVER

YAML / Config

```
import http from 'http'
const server = http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'applicat...
  res.end(JSON.stringify({ ok: true }))
})
server.listen(3000, () => console.log('listen...))
```

Most projects use Express/Fastify on top of http

06. STREAMS

Integration

```
import { createReadStream, createWriteStream ...
createReadStream('input.txt')
  .pipe(zlib.createGzip())
  .pipe(createWriteStream('output.gz'))
// Async iteration:
for await (const chunk of readableStream) {
  process(chunk)
}
```



SECURITY, MONITORING & OPERATIONS

Production hardening, observability, performance tuning & CLI reference

PART 2: OPS & SECURITY

07. ASYNC PATTERNS

Hardening

```
// Callback (legacy):
fs.readFile('a.txt', (err, data) => {})
// Promises:
const data = await fs.promises.readFile('a.tx...
// Parallel:
const [a, b] = await Promise.all([fetch1(), f...
// promisify legacy callbacks:
const readFile = util.promisify(fs.readFile)
```

10. CHILD PROCESSES

Unit Tests

```
import { exec, spawn } from 'child_process'
const { stdout } = await exec('ls -la')
const child = spawn('node', ['script.js'])
child.stdout.on('data', data => process.stdout...
child.on('exit', code => console.log(code))
```

spawn = streaming; exec = buffered; fork = for Node scripts

08. BUFFERS

Observability

```
const buf = Buffer.from('hello', 'utf8')
buf.toString('base64')
Buffer.alloc(256) // zero-filled buffer
buf.length / buf.slice(0, 4)
Buffer.concat([buf1, buf2])
```

Use for binary data, crypto, networking

11. COMMON GLOBALS

Commands

```
__dirname __filename // CJS only
import.meta.url // ESM
globalThis global // global object
setTimeout setInterval clearTimeout clearInte...
setImmediate() / process.nextTick()
console.log warn error time timeEnd
JSON.stringify / JSON.parse
```

09. PROCESS & ENV VARS

Optimization

```
process.env.PORT ?? 3000
process.env.NODE_ENV
process.argv // command-line args
process.exit(0) / process.exit(1)
process.cwd() // current working directory
process.on('uncaughtException', handler)
process.on('unhandledRejection', handler)
```

12. COMMON GOTCHAS

Warning

Single-threaded blocking I/O blocks ALL requests
Unhandled promise rejections crash process (Node 15)
process.nextTick runs before I/O events can starve I/O
require() caches modules mutations persist
ESM vs CJS interop: .mjs vs .cjs extensions or package.json type